

Note 5 — Dossier de projet ThermoSense

Module	Web Services — M2 Expert en Développement mobile & IoT
Évaluation	Note 5 — 40 % de la note finale — Commission d'architecture (Séance 9, 9 juin 2026)
Groupe	ThermoSense — Kenza · Enzo · Matteo · Tommy · Valentin
Dépôt	contrat-openapi.yaml · src/ · tests/ · bench/ · README.md · Rendu/
Démarrage	npm install && npm start → http://localhost:3000/api-docs · compte root/root

Ce dossier **synthétise** le fil rouge des séances 1 à 9. Il suit les **5 sections imposées** et renvoie aux notes détaillées pour les preuves. Les artefacts (contrat, code, tests, bench) **font foi** ; les chiffres cités sont reproductibles.

Sommaire et renvois : [A. Contrat & décisions](#) · [B. Sécurité](#) · [C. SOA/SOAP](#) · [D. Résilience](#) · [E. Bilan](#)

Notes sources : [01](#) / [02](#) (architecture & design), [03](#) / [04](#) (contraintes & scénarios), [05](#) / [06](#) / [10](#) (sécurité), [11](#) (SOA/SOAP), [12](#) (quality gate), [13](#) (remédiation audit), [14](#) (résilience).

Section A — Contrat API et décisions de design

A.1 Ressources et endpoints

API REST de supervision thermique : **hiérarchie** Building → Area (zone) → Sensor / Actuator / AlertThreshold , plus Measure (série temporelle d'un capteur) et User . Le contrat [contrat-openapi.yaml](#) (OpenAPI 3.0.3) expose **10 chemins** versionnés sous /v1 , tous documentés avec paramètres typés, schémas required , exemples métier réalistes et réponses d'erreur structurées Problem Details (application/problem+json , RFC 7807). Détail : [01-architecture-ressources.md](#) .

A.2 Évolution du contrat depuis la S1

Étape	Évolution	Preuve
S1–S2	Modèle de ressources, URIs, contraintes REST, premiers schémas	01 , 02
S3	Décisions de design avancé : pagination, idempotence (contrat), gestion d'erreurs	03 , 04
S4	Sécurité : threat model, RBAC/BOLA, matrice d'autorisations	05 , 10
S6–7	Audit croisé → remédiation : versioning /v1 , idempotence documentée , rate limiting, Problem Details, exemples. Score 4,5/8 → 8/8	11 , 13
S8	Résilience mesurée : découverte que l'Idempotency-Key est inerte serveur (14 doublons)	14
S9	Implémentation serveur de l'idempotence (eee5c31) → 0 doublon ; durcissement tests	14 §2.4 , tests/auth.test.js

A.3 Stratégie de versioning — par préfixe d'URI (/v1)

Versioning **par URI** (/v1 , /v2 , ...), synchronisé avec le `version` SemVer, appliqué sur les 3 serveurs déclarés (local, staging, prod). La politique d'évolution est **explicitée dans le contrat** (`info.description`) : évolutions **non-breaking** tolérées au sein de /v1 (ajout d'endpoint, de champ optionnel, d' `enum` toléré, de query param), évolutions **breaking** imposant /v2 (suppression/renommage, changement de type, optionnel→required, sémantique d'un code HTTP), **deprecation** ≥ 6 mois avec header `Sunset` (RFC 8594).

Pourquoi l'URI et pas le header ? L'URI est **visible dans les logs et le cache** (un proxy/CDN distingue /v1 de /v2 sans inspecter les headers), **triviale à router** côté passerelle, et **explicite pour un client mobile** déployé en masse (l'app épingle sa version). Ce qu'on **perd** : on « pollue » l'URI avec une préoccupation non-ressource, et un changement de version majeure change toutes les URLs (vs un simple header). **Pourquoi acceptable ici** : nos consommateurs sont des **apps mobiles versionnées** et des **devices IoT** au firmware figé — la stabilité et la lisibilité d'URL priment sur l'élégance REST pure. Le versioning par header (`Accept: application/vnd.thermosense.v2+json`) aurait été plus « pur » mais **invisible au cache** et plus fragile pour des clients embarqués.

A.4 Deux décisions de design difficiles

Décision 1 — Idempotence : **déduplication côté serveur** (et non « clé décorative »)

- **Problème.** L'ingestion `POST .../mesures` et le pilotage `PUT .../actuators/{id}` sont des **écritures à effet de bord** émises sur réseau mobile/IoT instable. Un timeout suivi d'un retry peut **dupliquer** la mesure (moyennes/seuils faussés) ou rejouer une commande.
- **Options.** (a) Rien (le client gère) ; (b) `Idempotency-Key` **documentée au contrat** mais non honorée ; (c) **déduplication serveur** : mémoriser `clé` → (statut, body) et rejouer la réponse sans ré-exécuter.
- **Choix & trajectoire.** Le contrat a d'abord retenu (b) en S6-7 (clé documentée). La **Note 4 (S8) a mesuré** que cette clé était **inerte** : à conditions réseau identiques, avec ou sans clé, **14 doublons** en scénario de référence — protection **décorative**. En **S9**, nous avons implémenté (c) : [src/idempotency.js](#) (clé `UUID` **obligatoire** → 422 sinon, rejeu identique → réponse mémorisée **24 h**, body différent → 409), câblé sur les deux écritures (`commit` → eee5c31).
- **Preuve.** Rejeu du bench : **0 doublon** sur les 3 scénarios (vs 2/14/107 en S8). Tests `tests/auth.test.js` (rejeu sans doublon, conflit 409, clé manquante 422).
- **Compromis assumé.** Store d'idempotence **en mémoire (mono-process)** → à porter sur store partagé (Redis) pour un déploiement multi-instances ; la clé devient **obligatoire** (un client non conforme est refusé).

Décision 2 — Refus d'accès BOLA : **404 plutôt que 403** (anti-énumération)

- **Problème.** Quand un `operator / reader` cible une ressource **hors de sa zone**, ou un `device` une ressource **autre que la sienne**, faut-il répondre `403 Forbidden` (« existe mais interdit ») ou `404 Not Found` (« n'existe pas pour vous ») ?
- **Options.** `403` est sémantiquement honnête mais **révèle l'existence** de la ressource (un attaquant énumère les IDs valides). `404` masque l'existence mais brouille le diagnostic légitime.
- **Choix.** **404** pour les violations **BOLA** (objet hors périmètre), **403** réservé aux violations **BFLA** (rôle/scope insuffisant sur la fonction). Implémenté dans [src/authorization.js](#) (`requireSensorAccess / requireActuatorAccess / requireAreaAccess` renvoient `denyNotFound`).

- **Compromis assumé.** Un développeur légitime qui se trompe de zone reçoit un `404` ambigu (il faut lire la matrice d'autorisations) — surcoût de DX accepté pour **ne pas offrir d'oracle d'énumération**.

Les 5 décisions de design fondatrices (URLs concises, zones indépendantes, seuils sous-ressource de zone, query params réservés à la lecture, JWT en header `Authorization`) sont détaillées dans [02-decisions-design.md](#) .

Section B — Dossier de sécurité

B.1 Threat model (synthèse)

Frontières de confiance : Client (mobile / console / device IoT) → **HTTPS + JWT** → API (middlewares `authN/authZ/validation/rate-limit` → handlers → données). Le détail (DFD, 3 endpoints analysés, matrice OWASP) est dans [05-threat-model.md](#) . **Risques prioritaires** retenus ([06-risques-prioritaires.md](#)) :

#	Risque (OWASP API)	Endpoints exposés	Contre-mesure implémentée
1	Broken Access Control (BOLA/BFLA)	tous les endpoints à <code>{id}</code>	RBAC 4 rôles + scopes (BFLA→403) et vérif d'appartenance zone/ressource (BOLA→404) — <code>src/authorization.js</code>
2	Excessive Data Exposure / volumétrie	GET <code>.../measures</code>	Pagination obligatoire (limit plafonné 499 , offset) — <code>src/routes/measures.js</code>
3	Injection / entrées invalides	POST/PUT à corps	Validation des champs + enum (state ∈ {on,off,auto}), erreurs 400 <code>invalidParameter</code> typées

Mécanismes transverses : **JWT** (HS256, `aud` / `iss` vérifiés, TTL access 30 min / refresh 7 j, `tokenType` contrôlé), **rate limiting** (100 req/min global, 10/15 min sur `/auth/login`), **Helmet** (en-têtes de sécurité), **journalisation de sécurité** ([src/security-logger.js](#) : échec auth, accès refusé, dépassement de quota).

B.2 Synthèse de l'audit (vulnérabilités corrigées)

Audit croisé reçu d'un groupe pair (S6-7) : **3 non-conformités bloquantes** + 4 mineures/majeures. Toutes traitées → score qualité **4,5/8 (56 %)** → **8/8 (100 %)** ([11 §2](#) , [13](#)). Bloquantes corrigées : **versioning** `/v1` (commit `2a1f340`), **idempotence** documentée puis **implémentée** (`3fe49b2` puis `eee5c31`), **rate limiting** (`3fe49b2`).

B.3 Preuves de tests

37 tests automatisés exécutables (`npm test`), couvrant happy paths **et cas adverses** :

- `tests/auth.test.js` (**27/27**) : RBAC par rôle, isolation BOLA inter-zones, ownership device, **tests adverses** (header absent, token expiré, signature invalide, `aud` non conforme, Bearer dupliqué), et **idempotence** (rejeu sans doublon, `409` conflit, `422` clé manquante).
- `tests/cadrage.test.js` (**10/10**) : questions de cadrage RBAC (création capteur/actionneur par rôle).

- Quality gate CI ([12](#)) : `spectral lint` (**0 erreur**), détection de secrets, `npm run test:auth / test:cadrage`, détection de breaking change.

B.4 Risques résiduels explicitement assumés

Risque résiduel	Pourquoi non traité	Mitigation future
Concurrence (ETag/If-Match) non implémentée	Conçue (scénario B, 04) mais le PUT /actuators ne vérifie pas If-Match → <i>last-write-wins</i> silencieux	Ajouter ETag + 412 Precondition Failed ; nécessite une persistance versionnée
Store en mémoire (données + idempotence)	Choix de démonstration (mono-process, ré-amorcé au boot)	BD relationnelle + contrainte d'unicité + store d'idempotence partagé
Secret JWT par défaut	Valeur de dev dans le code (JWT_SECRET)	Secret en variable d'env / coffre ; rotation ; envisager RS256
HTTPS non forcé côté serveur	Supposé géré par la terminaison TLS (reverse proxy)	HSTS, redirection 308, Strict-Transport-Security
Validation d'entrée partielle	Bornes métier (ex. température min/max) non systématiques	Schéma de validation (Joi/Zod) sur 100 % des écritures

Section C — Architecture SOA/SOAP

Décision : architecture hybride — SOAP *intégré*, de façon ciblée (et non écarté par principe). Détail : [11-note-3.md](#), extrait WSDL : [extrait-contrat-wsdl-note3.xml](#).

- **REST** pour le cœur IoT/mobile (capteurs, mesures, actionneurs, auth) : ~~95 % des volumes~~ (10 k mesures/min), clients mobiles, évolutivité, JSON natif. L'overhead XML/SOAP y serait contre-productif.
- **SOAP** pour **un** service : **BillingService** (facturation B2B des événements de seuil/maintenance). Trois critères justifient SOAP ici : **(1) interopérabilité legacy** — le partenaire impose son **WSDL** (ERP financier hérité) ; **(2) auditabilité / non-répudiation** — une facture est une pièce légale (conservation 10 ans), et **WS-Security** (XML-Signature/Encryption) répond nativement, là où REST n'a pas d'équivalent universel ; **(3) frontière métier** (bounded context DDD) avec cycle de vie propre.
- **Intégration** : le client mobile ne consomme **jamais** le SOAP directement — un **proxy REST→SOAP** (`POST /v1/invoices` → traduction WSDL) préserve l'UX mobile (1 hop + service de traduction).
- **Compromis assumés** : verbosité XSD (acceptée, élimine ~200 lignes de validation applicative) ; rigidité du versioning par namespace XML (acceptée car le partenaire évolue sur un cycle de 5-10 ans) ; **bus factor** SOAP/XSD (2 devs sur 4) → mitigé par isolation dans un microservice dédié et un runbook.

La **comparaison REST vs SOAP** appliquée au projet (formalisme du contrat, testabilité, évolutivité, complexité mobile, gouvernance) est détaillée dans [11-note-3.md §1](#).

Section D — Résilience mobile & IoT

Endpoint critique : `POST /v1/sensors/{id}/mesures` (ingestion IoT en entrepôt, 3G instable). Mesures **réelles et reproductibles** (`npm run bench`, seed `20260608`, N=100/passe) — dossier complet et

preuves : [14-note-4.md](#) , harness [bench/](#) .

D.1 Patterns implémentés et configuration retenue

Pattern	Configuration	Justification	Compromis
Timeout différencié	write 3 000 ms / read 1 500 ms	Borne le gel ; une ingestion tolère plus de latence qu'un read bloquant l'UI	Un write lent légitime (> 3 s) compté en échec → retry
Retry borné + backoff exp. + full jitter	max 3 , base 500 ms , cap 4 000 ms	Récupère les pertes transitoires (+18 pts en réf. B) ; le jitter évite le <i>thundering herd</i> à la reprise réseau	+req/min (réseau/batterie), +latence de queue
Idempotency-Key (déduplication serveur)	UUID obligatoire, fenêtre 24 h , src/idempotency.js	Rend le retry sûr sur écriture : 0 doublon mesuré	Store en mémoire (mono-process) ; clé obligatoire
Dégradation gracieuse (gestion d'erreurs)	Codes Problem Details stables ; 503 si store non prêt ; pas de 500 sur erreur client	Le client mobile branche un fallback déterministe par code	—

Où changer le retry sans redéployer le serveur ? Les paramètres de retry/timeout sont **côté client** (*bench/degraded-client.js* , et dans l'app mobile réelle) — un changement de stratégie de retry n'impose **aucun** déploiement serveur ni nouvelle version d'API. Les quotas serveur, eux, sont pilotés par **variables d'environnement** (*RATE_LIMIT_MAX* , ...) sans changement de code.

D.2 Résultat clé — avant/après (réf. B, 20 % de perte)

Métrique	Baseline (conforme, sans résilience)	Après (timeout+retry, clé honorée)	Verdict
Taux de succès	82 %	100 %	✅ +18 pts
p95 latence perçue	8 000 ms (gel)	4 913 ms	✅ gel éliminé
Doublons persistés	0	0	✅ (était 14 en S8, clé inerte)
Consommation réseau	60 req/min	74 req/min	+23 % (coût)

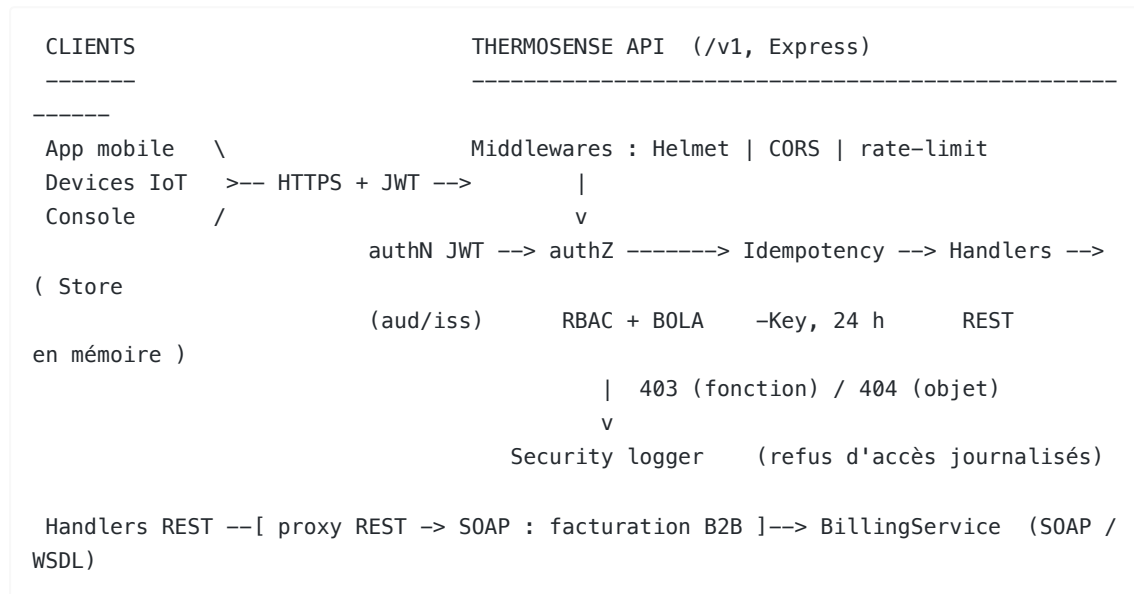
D.3 Limites restantes

- **Disponibilité sous panne sévère (scénario C, 50 % de perte)** : même conforme + résilient, le succès plafonne à **66 %**, la p95 monte à **14 s** et la charge réseau **triple** (170 req/min). Le retry client **seul** ne suffit pas → il faut une **file locale + backpressure** (conçue, **non implémentée** faute de temps). L'**intégrité** est acquise (0 doublon) ; la **disponibilité** est le prochain chantier.

- **Retry non adaptatif** : sur bon réseau (scénario A), le retry agressif est **net négatif** (aucun gain de latence, +req) → une activation **conditionnelle** aux conditions mesurées serait préférable.
- **Écart contrat/implémentation 429** : le contrat spécifie `rateLimitExceeded` + `Retry-After` ; l'implémentation renvoie `tooManyRequests` + en-têtes `RateLimit-*` ([14 §3.4](#)).

Section E — Bilan d'architecture et recommandations

E.1 Schéma d'architecture final



Frontières : tout passe par **HTTPS + JWT** ; l'autorisation est **systématiquement à deux étages** (fonction puis objet) ; les écritures à effet de bord traversent le **garde-fou d'idempotence** ; la facturation B2B est **isolée** derrière un proxy vers le service SOAP.

E.2 Trois décisions que nous prendrions différemment

1. **Implémenter l'idempotence serveur dès qu'on la documente au contrat** (S6-7), pas un sprint plus tard. Documenter une garantie sans l'implémenter a créé une **protection décorative** que seule la mesure (S8) a révélée. *Leçon : une clause de contrat à effet de sécurité doit être livrée avec son test.*
2. **Choisir une persistance versionnée (BD + contraintes) plus tôt.** Le store en mémoire a simplifié la démo mais a **bloqué** la concurrence (ETag/If-Match conçue mais non implémentable proprement) et imposé un store d'idempotence mono-process. C'est la décision à plus fort **effet de levier** rétrospectivement.
3. **Valider le contrat et l'implémentation ensemble** (tests de contrat type Schemathesis). L'écart 429 (contrat vs `express-rate-limit`) aurait été détecté tôt par un test de conformité, pas en relecture.

E.3 Axes prioritaires pour un passage en production

1. **Persistance réelle** (BD + unicité) → débloquent concurrence, idempotence partagée, durabilité.
2. **Disponibilité offline-first** : file locale + backpressure côté client (règle le scénario C) ; retry adaptatif.
3. **Durcissement sécurité** : secret JWT hors-code + rotation (voire RS256), HTTPS forcé (HSTS), validation d'entrée exhaustive, scan de dépendances bloquant en CI.

4. **Observabilité** : métriques (taux d'erreur, p95, doublons évités), traçage des refus d'accès, alerting.
5. **Aligner le contrat 429** et activer en CI les règles Spectral custom + `oasdiff` (anti-breaking).

Ce que la commission cherche : non pas un projet sans défaut, mais des **compromis assumés avec des raisons techniques**. Notre fil conducteur — mesurer avant de décider — a transformé une garantie décorative (idempotence inerte) en garantie prouvée (0 doublon), et a **déplacé** le risque résiduel vers un problème clairement nommé (disponibilité sous panne), avec son chemin de résolution.